

Praetor — A Mini Online Judge

Software Requirements Specification (SRS)

K S Sazid Abesh

Version 1.0.0

Table of Contents

Cover

Project Title	Praetor — A Mini Online Judge
Course	CSE470 — Software Engineering
Institution	BRAC University, Department of CSE
Author(s)	K S Sazid Abesh
Document Version	1.0.0

1. Introduction

1.1 Type of Project

Praetor is a **web-based online judge** — an automated code-evaluation and competitive-programming platform. A user submits source code for a problem; the system compiles and runs it inside an isolated sandbox against a set of hidden test cases, then returns a verdict (Accepted, Wrong Answer, Time Limit Exceeded, etc.). Around this core it provides timed contests with live standings, in the style of a small, self-hosted Codeforces.

1.2 Purpose

The purpose of Praetor is to give programmers a safe, automated environment to practice algorithmic problems and compete in contests, while giving problem setters and administrators the tools to author problems, run contests, and maintain integrity. The system removes manual grading by executing submissions deterministically against predefined test cases under enforced time and memory limits.

1.3 Target Users

- **Contestant / Coder** — submits solutions, views verdicts, joins contests, tracks rating and statistics.
- **Problem Setter** — authors problems, uploads test cases, attaches editorials.
- **Administrator** — manages contests, rejudges submissions, posts announcements, oversees the platform.

1.4 Scope

The system covers user accounts and roles, problem and test-case management, sandboxed multi-language judging, timed contests with ICPC-style scoring and live standings, ratings and user statistics, and administrative controls. Code execution is isolated per submission. The platform is delivered as containers and runs locally via `docker compose up`; cloud hosting and horizontal scale-out (Kafka, Kubernetes) are optional extensions.

2. Functional Requirements

Priority: **H** = high (core), **M** = medium, **L** = low/stretch. Authentication, registration, roles and profile are baseline requirements (FR-1–FR-3) and are not counted toward the 24 domain features (FR-4–FR-27).

2.1 Baseline — Accounts & Access *(not counted as features)*

ID	Requirement	Actor	Priority
FR-1	The system shall allow a user to register with handle, email and password.	Guest	H
FR-2	The system shall authenticate users and issue a JWT on login.	User	H
FR-3	The system shall enforce role-based access (Coder, Setter, Admin) and expose a profile.	User	H

2.2 Judging Engine

ID	Requirement	Actor	Priority
FR-4	The system shall accept a code submission and execute it in an isolated sandbox, returning a verdict.	Coder	H
FR-5	The system shall support judging in multiple languages (C++, Python, Java).	Coder	H
FR-6	The system shall report a per-test-case verdict (AC/WA/TLE/MLE/RE/CE).	Coder	H
FR-7	The system shall enforce and report per-submission time, memory and process limits.	System	H
FR-8	The system shall queue submissions asynchronously with status QUEUED → JUDGING → DONE.	System	H
FR-9	The system shall capture and display compilation errors.	Coder	H
FR-10	The system shall let a user view their submission history and source code.	Coder	M

ID	Requirement	Actor	Priority
FR-11	The system shall support special/float judging (tolerance, multiple valid answers).	Setter	M

2.3 Problem Service

ID	Requirement	Actor	Priority
FR-12	The system shall allow a setter to create, edit and delete problems.	Setter	H
FR-13	The system shall allow bulk upload of test cases (sample and hidden).	Setter	H
FR-14	The system shall support difficulty ratings and tags on problems.	Setter	M
FR-15	The system shall allow searching and filtering problems by tag, difficulty and text.	User	M
FR-16	The system shall allow attaching an editorial/solution to a problem.	Setter	L

2.4 Contest

ID	Requirement	Actor	Priority
FR-17	The system shall allow an admin to create a contest with a time window and problem set.	Admin	H
FR-18	The system shall display live standings updated in real time.	User	H
FR-19	The system shall compute ICPC-style scoring with penalty.	System	H
FR-20	The system shall allow users to register for a contest.	Coder	M
FR-21	The system shall freeze the standings during the last N minutes.	System	M
FR-22	The system shall support virtual participation / upsolving after a contest ends.	Coder	M
FR-23	The system shall deliver live clarifications and announcements during a contest.	User/Admin	M

2.5 Identity & Insights

ID	Requirement	Actor	Priority
FR-24	The system shall maintain a user rating (ELO) and global rank, updated after contests.	System	M
FR-25	The system shall present per-user solve statistics and an activity heatmap.	User	M
FR-26	The system shall rate-limit submissions to prevent spam/abuse.	System	M

2.6 Administration & Client

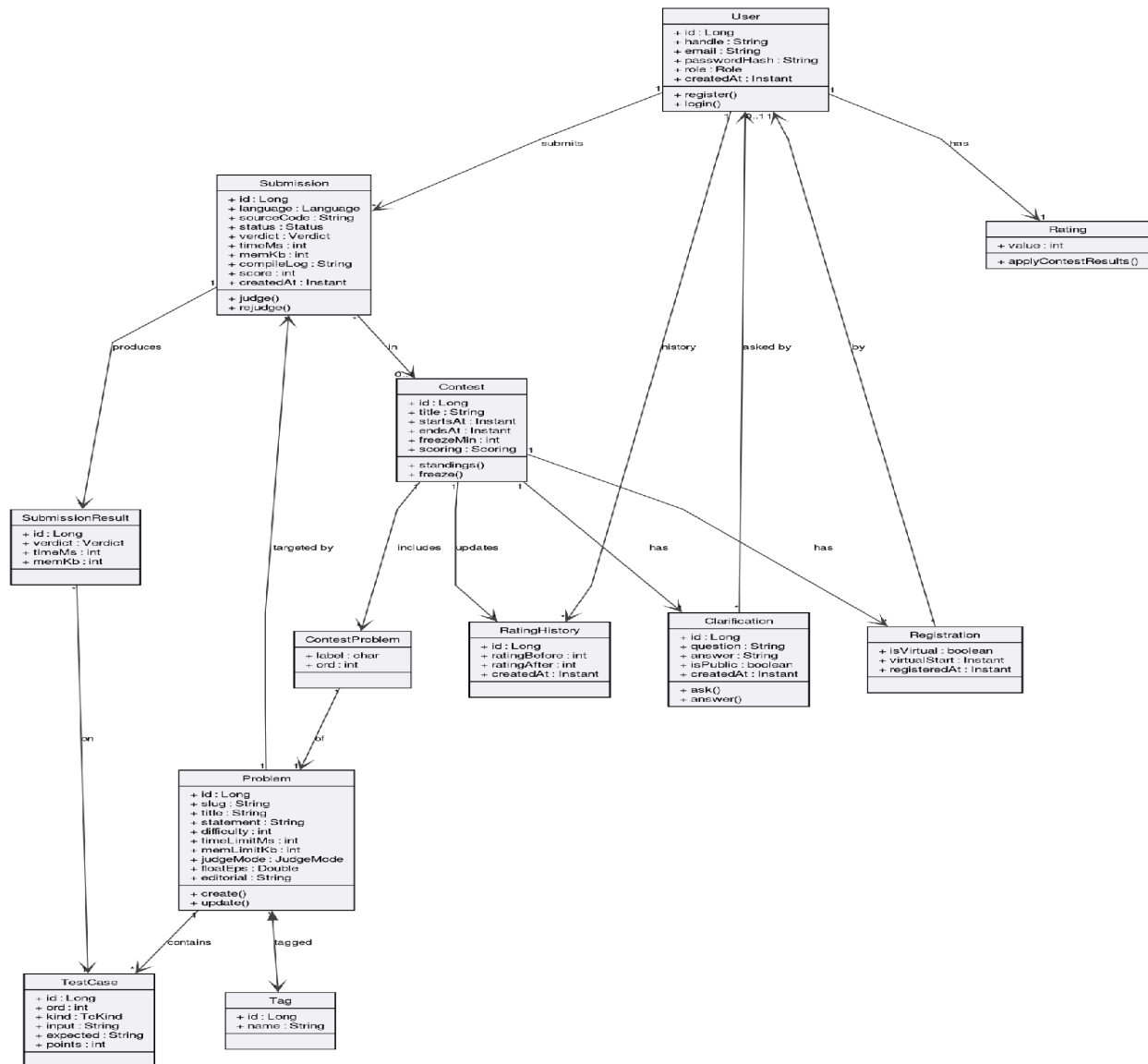
ID	Requirement	Actor	Priority
FR-27	The system shall allow an admin to rejudge a submission and recompute standings.	Admin	M

3. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	A submission verdict shall typically return within a few seconds for standard problems; the judge processes submissions concurrently via a worker pool.
NFR-2	Security (isolation)	Submitted code shall run in an isolated container with no network access and strict CPU/memory/PID/time limits; containers are destroyed after each run. Untrusted code must not affect the host or other submissions.
NFR-3	Security (access)	All write endpoints shall validate the user's role server-side; JWTs shall expire; inputs shall be validated and database access parameterised.
NFR-4	Reliability	The submission queue shall recover gracefully on restart; no accepted submission is silently lost.
NFR-5	Usability	The web client shall provide a code editor with syntax highlighting and clear, immediate verdict feedback.
NFR-6	Scalability	The judging stage shall be decoupled so judge workers can be scaled horizontally (optional Kafka consumer group + Kubernetes replicas).
NFR-7	Portability	The entire system shall start from <code>docker compose up</code> on any Docker-capable host.
NFR-8	Maintainability	The backend shall follow an MVC layering (Controller → Service → Repository) and ship with automated tests.
NFR-9	Integrity	Hidden test cases shall never be exposed to contestants through any API.

4. Class Diagram

The domain model below is generated with software (Graphviz) and mirrors the database schema. It shows the core entities, their attributes, key operations, and the relationships (with multiplicities) between them.



Praetor domain class diagram

Description. A User submits many Submissions, each targeting one Problem and optionally belonging to a Contest. A Problem owns many TestCases and may carry many Tags. Judging a Submission produces a SubmissionResult per TestCase. A Contest includes problems (ContestProblem), accepts Registrations from users, carries Clarifications, and on completion writes RatingHistory and updates each user's Rating.

5. Tools and Technologies

5.1 Frontend

- **Angular** (TypeScript) — single-page application, component-based MVC view layer.
- **Monaco Editor** — in-browser code editor with syntax highlighting.
- **STOMP over WebSocket** — live standings and clarifications.

5.2 Backend

- **Spring Boot (Java)** — REST controllers, services, WebSocket endpoints.
- **PostgreSQL** — relational data store.
- **Docker Engine** — per-submission sandbox using cgroups v2 for CPU/memory/PID/time limits.
- **Spring WebSocket** — real-time push to clients.

5.3 Build, Test & Tooling

- **Maven** — backend build.
- **JUnit + Testcontainers** — unit and integration tests against a real PostgreSQL.
- **springdoc / OpenAPI (Swagger UI)** — API documentation.
- **Docker Compose** — local orchestration of all services.

5.4 Optional / Future

- **Apache Kafka** — durable submission queue (consumer group of judge workers).
- **Kubernetes (Minikube)** — containerised deployment with scalable judge-worker replicas.

6. Compatibility with System Environment / OS

Praetor is delivered as Docker containers, so it runs on any host with Docker Engine and Docker Compose:

- **Linux** — the primary and recommended environment. The sandbox relies on **cgroups v2** for resource limiting, which Linux provides natively; this is where judging is most accurate and efficient.
- **Windows / macOS** — supported through **Docker Desktop**, which runs the containers inside a Linux VM. The full stack runs unchanged; resource limiting works via the Docker Desktop Linux backend.
- **Client** — the web application runs in any modern browser (Chromium, Firefox, Edge); no installation is required on the client side.
- **Database & services** — PostgreSQL, the Spring Boot backend and the Angular frontend all run as Compose services on an internal network; only the gateway/UI port is exposed to the host.

A single `docker compose up` brings up the database, backend and frontend together, applying schema and seed data on first boot.

7. Implementation

Screenshots are added during/after the demonstration. Each feature's intended screen and a one–two sentence description are listed below.

7.1 Registration & Login

[SCREENSHOT: registration and login forms] Users create an account and sign in; a JWT is issued and stored for authenticated requests.

7.2 Problem List & Search

[SCREENSHOT: problem list with tag/difficulty filters] The catalogue of problems with search and filtering by tag, difficulty and text.

7.3 Problem Detail

[SCREENSHOT: problem statement page] Full statement, constraints, limits and sample cases for a single problem.

7.4 Code Submission

[SCREENSHOT: Monaco editor with language picker and submit button] The contestant writes or pastes code, selects a language, and submits it for judging.

7.5 Verdict & Per-Test Results

[SCREENSHOT: verdict page showing per-test-case outcomes] The overall verdict plus a per-test-case breakdown with time and memory used.

7.6 Compilation Error Display

[SCREENSHOT: compile error output] When code fails to compile, the compiler message is shown to the user.

7.7 Submission History

[SCREENSHOT: list of past submissions + code viewer] A user's past submissions with status, verdict and the ability to view the source.

7.8 Problem Authoring (Setter)

[SCREENSHOT: create/edit problem form] Setters create and edit problems, including limits and judge mode.

7.9 Bulk Test-Case Upload

[SCREENSHOT: test-case upload screen] Setters upload sample and hidden test cases in bulk for a problem.

7.10 Editorial

[SCREENSHOT: editorial attached to a problem] An optional editorial/solution is attached and displayed after solving.

7.11 Contest Creation

[SCREENSHOT: contest creation form] Admins define a contest's time window, scoring and problem set.

7.12 Contest Registration

[SCREENSHOT: register button on contest page] Users register for an upcoming or running contest.

7.13 Live Standings

[SCREENSHOT: real-time scoreboard] The scoreboard updates live over WebSocket as submissions are judged.

7.14 Standings Freeze

[SCREENSHOT: frozen scoreboard near contest end] Standings freeze during the final minutes to preserve suspense.

7.15 Virtual / Upsolve

[SCREENSHOT: virtual participation screen] Users replay a finished contest virtually or upsolve unsolved problems.

7.16 Clarifications & Announcements

[SCREENSHOT: clarifications panel] Live questions and admin announcements during a contest.

7.17 Rating & Global Rank

[SCREENSHOT: rating page / leaderboard] Per-user ELO rating and the global ranking of all users.

7.18 Solve Statistics & Heatmap

[SCREENSHOT: stats page with activity heatmap] Solved/attempted counts, accuracy, verdict breakdown and an activity heatmap.

7.19 Submission Rate-Limiting

[SCREENSHOT: rate-limit message on rapid submit] Rapid repeated submissions are throttled to prevent abuse.

7.20 Admin Rejudge

[SCREENSHOT: rejudge action in admin view] Admins rejudge a submission and recompute affected standings.

8. Challenges

- **Sandbox security.** Running arbitrary, untrusted code safely is the hardest requirement: it demands strict isolation (no network, capped CPU/memory/PIDs/time) and reliable cleanup so a malicious or runaway submission cannot harm the host or other users.
- **Concurrency & asynchronous judging.** Submissions must be queued and processed by a worker pool without blocking the API, while accurately tracking each submission's lifecycle.
- **Real-time state synchronisation.** Live standings must stay consistent across many browsers, including clients that disconnect and reconnect mid-contest.
- **Contest timing.** Countdown timers must remain correct despite client/server clock differences (solved by sending absolute end-times).
- **Team ramp-up.** Building deep Spring Boot and Angular features by hand within the sprint timeline, alongside learning the stack.

9. Conclusion

Praetor delivers a working, self-hosted online judge: secure sandboxed multi-language judging, problem and test-case management, timed contests with live ICPC standings, and user ratings and statistics — all runnable locally with a single command. Beyond meeting the course's functional scope, the project builds practical depth in process isolation, concurrency, real-time systems and full-stack engineering with Spring Boot and Angular. Future work includes additional judging languages and optional horizontal scale-out of judge workers via Kafka and Kubernetes.

10. References

1. IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
2. Spring Boot Documentation — <https://spring.io/projects/spring-boot>
3. Angular Documentation — <https://angular.dev>
4. PostgreSQL Documentation — <https://www.postgresql.org/docs/>
5. Docker Documentation — <https://docs.docker.com/>
6. Codeforces — <https://codeforces.com> (reference for online-judge behaviour and ICPC scoring).
7. WeHeal — CSE470 Project SRS (course sample document).